

Infrastructure Mastery Manual: Vol 2

Technical Reference: Registry, Meta-Arguments, & Networking

Section 1: Terraform Registry & EC2 Engineering

The Registry is the authoritative source for provider schemas. When provisioning EC2, we must look beyond basic launch specs toward security and compliance.

```
# Implementing Security-First Compute
resource "aws_instance" "app_compute" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t3.medium"

  # IMDSv2 Enforced to prevent SSRF
  metadata_options {
    http_endpoint = "enabled"
    http_tokens   = "required"
  }

  root_block_device {
    encrypted = true
    volume_size = 50
  }
}
```

Section 2: Terraform Meta-Arguments

Meta-arguments allow granular control over resource lifecycles. Engineers should prefer `for_each` over `count` to preserve index stability, as index shifts cause unnecessary resource replacement.

```
# Stable Key Engineering with for_each
resource "aws_instance" "node" {
  for_each      = var.instances # Map of objects
  ami           = "ami-0123"
  instance_type = each.value.size
  tags          = { "Name" = each.key }

  lifecycle {
    prevent_destroy = true # Protect Production DBs
    create_before_destroy = true # Reduce downtime
  }
}
```

Section 3: Network Components Architecture

A production-grade VPC requires isolation via public/private subnet separation and high-availability NAT gateways.

```
# High Availability Networking Pattern
resource "aws_nat_gateway" "main" {
  for_each      = aws_subnet.public
  allocation_id = aws_eip.nat[each.key].id
  subnet_id     = each.value.id

  # Ensure IGW exists before NAT
  depends_on = [aws_internet_gateway.main]
}

# Standardizing Route Tables
resource "aws_route_table_association" "private" {
  for_each      = aws_subnet.private
  subnet_id     = each.value.id
  route_table_id = aws_route_table.private[each.key].id
}
```